IAM Roles, Security Tools, and Monitoring

Explore how IAM roles enable secure, temporary access to AWS resources and understand best practices for their use. Learn how AWS security tools such as CloudTrail, IAM Access Analyzer, and GuardDuty help monitor, audit, and protect your applications by detecting suspicious activity and misconfigurations. This lesson equips you to implement secure access with temporary credentials while maintaining robust security governance and compliance.

In this lesson, we'll explore how AWS enables secure and temporary access to resources using IAM roles, and how developers can monitor and secure that access using AWS security services and observability tools. We'll start with the problem IAM roles solve, examine how temporary credentials work, and conclude how services like CloudTrail, IAM Access Analyzer, and GuardDuty enhance visibility and governance.

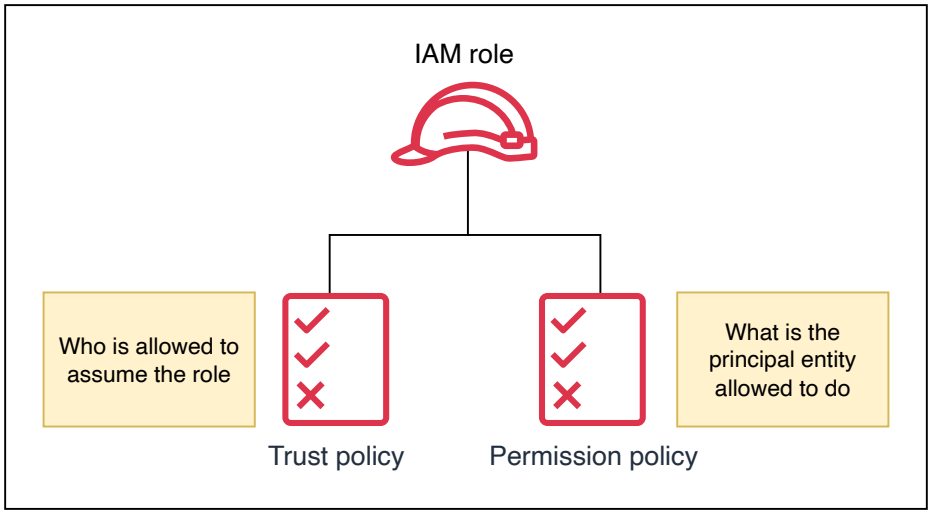
IAM roles

IAM roles are identities that AWS services or applications can assume to perform actions on AWS resources. IAM roles provide secure, temporary access to AWS resources without long-term credentials, supporting scalable and secure application development.

Ask

They are used whenever an AWS service or external user needs to perform actions without using static credentials, such as a Lambda function accessing an S3 bucket. Roles are created with permissions and can be assumed by trusted entities. Multiple policies associated with an IAM role dictate the scope of permissions of the role and the entities that are allowed to use the role. These policies are as follows:

- **Trust policy:** This resource-based policy specifies the entities allowed to assume an IAM role.
- **Permissions policy:** This identity-based policy specifies the role's permissions.



Policies attached to IAM roles

An IAM role can have multiple permission policies attached to it, but only one trust policy. Also, we can create an IAM role without a permission policy, but specifying a trust policy is required during the creation of an IAM role. Upon assuming a role, AWS issues temporary credentials enabling secure access within the permission boundaries defined by the role.

Ask

Using MFA for secure role assumptions

To enhance IAM role security, AWS allows roles to require multi-factor authentication (MFA) during the `AssumeRole` process. This is enforced in the trust policy using a condition like `"aws:MultiFactorAuthPresent": "true"`.

This ensures that even if access keys are compromised, attackers can't assume roles without an MFA token. It's particularly useful for roles with administrative privileges or those used in cross-account access.

Understanding the basic idea of IAM roles provides the foundation for understanding how AWS applies them in a broader security model known as role-based access control (RBAC).

Role-based access control (RBAC) with IAM roles

AWS Identity and Access Management (IAM) roles implement role-based access control (RBAC) principles. RBAC allows centralized management of permissions, simplifying security administration and enforcing the principle of least privilege. It is used when multiple users, services, or applications need standardized access to resources based on their roles rather than individual identity management. Permissions are attached to roles, not users. Services and applications assume roles to obtain the permissions they need, ensuring consistent, auditable, and scalable access management.

To understand how roles provide temporary permissions securely, we need to explore the nature of the credentials that AWS issues when a role is assumed.

Ask

Understanding temporary credentials

Temporary credentials prevent the risks associated with static access keys by providing short-term, automatically expiring access to AWS resources. They are used whenever a service or user assumes a role, allowing AWS to authorize actions securely for a limited time. Temporary security credentials include an access key ID, a secret access key, a session token, and an expiration date and time. These credentials expire automatically and act as bearer tokens during their active session for securely authenticating API requests.

Temporary credentials must be issued and managed properly, which leads us to the next critical service: AWS Security Token Service (STS).

How AWS STS issues temporary credentials

AWS Security Token Service (STS) provides a centralized and secure way to issue and manage temporary credentials for AWS roles. STS is used when services, applications, or federated users need temporary, scoped access to AWS resources, such as cross-account access. When the `GetSessionToken` API is called, STS validates the request, issues temporary credentials with associated policies, and provides them back to the caller for secure, temporary use.

Ask

Using IAM role to gain temporary access to AWS services

Once we understand how temporary credentials are issued, the next step is knowing how applications typically obtain and use these credentials programmatically.

Using IAM roles in application code

Applications often need dynamic, scoped access to AWS resources without relying on hardcoded or long-term credentials. IAM roles allow us to securely grant temporary access to these services using short-lived credentials.

For example, an application running locally can call the `AssumeRole` API using the AWS SDK, retrieve temporary credentials, and use those to interact with AWS services. This enables secure access delegation across accounts or for specific tasks.

Below, we create both S3 and EC2 clients using the credentials from an assumed role. The role has permissions for S3 only. As expected, the S3 call succeeds, but the EC2 call results in an access denied error:

Ask

main.py Python 3.10.4

```
import boto3

sts = boto3.client('sts')
creds = sts.assume_role(
    RoleArn='arn:aws:iam::123456789012:role/S3ReadOnlyRole',
    RoleSessionName='AppSession'
)['Credentials']

s3 = boto3.client(
    's3',
    aws_access_key_id=creds['AccessKeyId'],
    aws_secret_access_key=creds['SecretAccessKey'],
    aws_session_token=creds['SessionToken']
```

Using IAM role for AWS access in an application

This highlights the security advantage of using dedicated roles for specific access; an application with access to one service (S3) cannot misuse permissions to another (EC2).

Equally important is the temporary nature of these credentials. Every `AssumeRole` call issues credentials valid only for a limited time (typically 15 minutes to 1 hour). This reduces the risk in case credentials are accidentally exposed, they expire quickly, limiting potential misuse.

If our application already has an identity token from an external identity provider (like Cognito or OIDC), we can use `AssumeRoleWithWebIdentity` instead. This lets our app exchange the token for temporary credentials without using any access keys:

main.py Python 3.10.4

```
sts = boto3.client('sts')
creds = sts.assume_role_with_web_identity(
    RoleArn='arn:aws:iam::123456789012:role/WebAppAccessRole',
    RoleSessionName='WebSession',
    WebIdentityToken='token-from-idp'
```

Ask

```
    )['Credentials']
```

Using IAM role for AWS access in an application

> **Note:** `AssumeRoleWithWebIdentity` does not require AWS access keys, making it ideal for mobile, browser-based, or containerized apps that authenticate using identity providers.

These temporary credentials are automatically rotated and expire after a set time window, providing a secure and auditable way to grant access to AWS services without long-term secrets.

Using IAM roles with AWS services

To securely interact with AWS services like Lambda, DynamoDB, or S3 from our APIs, we should use IAM roles rather than embedding credentials. IAM roles allow services to assume predefined permissions temporarily and securely.

Consider a serverless web application where the back-end Lambda functions need to read data from a DynamoDB table. Instead of embedding static access keys into the code, we assign the Lambda function an execution role that grants read permissions. When the Lambda function executes, it automatically assumes the role and securely accesses the DynamoDB table with temporary credentials issued by AWS STS.

Ask

Application using IAM role to access DynamoDB

Through such real-world implementations, we see how IAM roles simplify security management while enabling dynamic, scalable applications.

Best practices for managing roles

Following best practices ensures access control remains secure, auditable, and compliant with organizational security policies:

- Assign minimum necessary permissions to roles.
- Set the shortest practical expiration times for temporary credentials.
- Monitor role usage with AWS CloudTrail, which logs all role-related operations.
- Regularly review and rotate role permissions.

IAM roles are indispensable for modern AWS development, allowing us to grant secure, temporary, and least-privilege access to AWS services. By understanding why IAM roles exist, when to use them, and how services assume roles th STS-issued credentials, we build more secure and maintainable AWS applications.

Ask

Secure access practice

As discussed earlier, our applications often interact with AWS services like Amazon S3, DynamoDB, or SNS. A common but unsecure approach is to hardcode credentials directly into the application code. This creates a significant security risk; if the source code is leaked or improperly shared, attackers can gain unauthorized access to AWS resources.

Let's first look at an example of this unsecure practice:

main.py Python 3.10.4

```
import boto3

# BAD PRACTICE: Hardcoded AWS credentials should never be used in sou
# Use IAM roles (if running on AWS) or configuration files/environmen

s3_client = boto3.client(
    's3',
    aws_access_key_id='<Sample Access ID>',           # ❌ Avoid hard
    aws_secret_access_key='<Sample secret access key>'  # ❌ Sensitiv
)

# List S3 buckets (works, but unsecure if credentials are exposed)
response = s3_client.list_buckets()
```

Unsecure code

Hardcoding AWS credentials into source code poses a serious security risk. If the code is exposed or shared improperly, these credentials can be used to access AWS resources maliciously.

Instead, we can use temporary credentials generated using the `GetSessionToken` API. This allows developers to request time-limited credentials from AWS S using their existing IAM user credentials, without embedding those long-term keys in the application.

Ask

Here is an example of the recommended approach:

main.py Python 3.10.4

```
import boto3
import botocore

# Long-term credentials should be stored securely (e.g., environment
session = botocore.session.get_session()
sts = session.create_client('sts')

# Get temporary session token
response = sts.get_session_token()

# Use the temporary credentials
credentials = response['Credentials']
s3 = boto3.client('s3'
```

Secure code

By using temporary credentials, we eliminate the need to embed long-term access keys into application code, enhancing both the security and flexibility of our AWS applications.

Now that we understand why secure, flexible access management is important. Let's take a closer look at how IAM roles, policies, and temporary credentials work together to protect our applications.

Monitoring and governing

Using roles securely is only the first step. We also need to monitor who assumed what role, when, and why. AWS offers several tools to audit and protect IAM usage.

Ask

Auditing role usage

AWS CloudTrail is the foundational tool for monitoring and auditing IAM activity. Every time a role is assumed—whether by a Lambda function, an EC2 instance, or a user—CloudTrail captures the `AssumeRole` API event. It logs critical metadata such as the identity of the caller, the time of the request, the source IP address, and the AWS service being accessed.

For example, if a developer's IAM user assumes a role that grants access to production data, CloudTrail will log the following:

- Who assumed the role (principal identity)?
- Which role was assumed?
- When the event occurred?
- What resources were accessed using that role?

These logs are essential for security audits, incident investigations, and compliance checks. We can store CloudTrail logs for analysis and to create alerts.

CloudTrail is your first tool for finding out who did what, when, and how.

Identifying unintended access

Even with least-privilege policies, IAM misconfigurations are easy to make—especially in large environments with multiple accounts, services, and developers. IAM Access Analyzer helps identify these risks by scanning IAM policies to determine whether they grant access to external entities, including:

Ask

- Users and roles from other AWS accounts.
- AWS services that shouldn't have access.

- Anonymous or public access.

For example, if a developer mistakenly adds `"Principal": "*"` to a trust policy, making a role assumable by anyone, IAM Access Analyzer will flag this configuration as a finding. The tool also integrates with the AWS Organizations framework, making it useful for multi-account governance.

This proactive analysis helps answer the question: "*Is there a path for external access that shouldn't exist?*"

Reviewing credential usage

While IAM roles themselves don't have long-term credentials, understanding who is assuming them, and whether they're following best practices, still matters. The **IAM credential report** gives a high-level overview of credential activity for all IAM users in the account. This helps indirectly govern role usage, especially when users are the ones assuming roles.

For example, if a user frequently assumes a privileged role but doesn't use MFA or hasn't rotated their access keys in months, that's a potential security risk.

We can generate the credential report from the AWS CLI by executing the following command:

main.sh

```
aws iam generate-credential-report
aws iam get-credential-report --output text --query Content | base64 -
```

Generate IAM credential report

The report doesn't log role assumptions directly—but it helps enforce a secure foundation for the identities assuming those roles. For actual role usage activity (such as who assumed a role and when), we need to rely on AWS CloudTrail, which records `AssumeRole` events.

Identifying suspicious behavior

GuardDuty adds a threat-analysis layer on top of audit logs to detect suspicious or unexpected activity. It continuously monitors data sources such as:

- AWS CloudTrail (including `AssumeRole` events)
- VPC Flow Logs (network activity)
- DNS logs (name resolution activity)

If GuardDuty sees an IAM role being assumed from an unusual IP address, to call rarely accessed services, or invoked in rapid succession in multiple regions, it will generate a security finding. For example:

Ask

- A known malicious IP assuming a trusted role
- A role being used to enumerate S3 buckets in a pattern resembling data exfiltration
- Credential usage from a geolocation we've never operated in

GuardDuty findings are scored based on severity and integrated into AWS Security Hub and EventBridge, enabling automated responses like revoking

Ask